

Part I

Q1) A DBMS stores and manages data in a structured way, providing security, backup, recovery, and support for multiple users. A traditional file system stores data in separate files and lacks these features. A DBMS reduces data duplication and maintains consistency, while a file system may lead to redundant and inconsistent data.

Scenario : A college management system that stores student records, course registrations, grades, and faculty information. A DBMS is the better choice because many departments need to access and update the same data. It ensures data consistency, prevents duplication, provides security, and allows efficient querying of information.

Q2) A strong entity has its own primary key and can be uniquely identified without depending on any other entity. A weak entity does not have a complete primary key of its own and depends on a strong entity for identification.

The primary key of a weak entity is formed by combining:

1. The primary key of its owner (strong) entity, and
2. Its own partial key (also called a discriminator).

The relationship that connects a weak entity to its owner entity is called an identifying relationship. In an ER diagram, a weak entity is represented by a double rectangle and the identifying relationship by a double diamond.

Q3) The phone number should be modeled as a multi-valued attribute. This is because a student can have more than one phone number. A simple attribute can store only a single value, while a composite attribute is used when an attribute can be divided into smaller parts (for example, address into street, city, and state). So, it can't be modeled as a simple or composite attribute. In an ER diagram, a multivalued attribute is shown using a double ellipse.

Q4) A super key is any set of one or more attributes that can uniquely identify a tuple in a relation. A candidate key is a minimal super key, meaning that no attribute can be removed from it without losing the property of unique identification. Thus, every candidate key is a super key, but not every super key is a candidate key.

For the relation Student(roll_no, email, name, branch), since roll_no of a student will always be unique and assuming email is also unique:

Candidate Keys: roll_no, email

These are candidate keys because each can uniquely identify a student and they also don't contain any unnecessary attributes.

Examples of Super Keys that are not Candidate Keys:

- (roll_no, name)
- (roll_no, branch)
- (email, name)
- (email, branch)
- (roll_no, email)

These are super keys because they can uniquely identify a student, but they are not candidate keys because they contain extra attributes and are therefore not minimal.

Q5)

- Referential Integrity Constraint.
- It protects against: It ensures that every student_id in the Enrollment table belongs to a student who actually exists in the Student table.
- The insert operation should be rejected (or fail) until a valid student_id exists in the Student table.

Q6)

- A separate relationship (junction/associative) table.
- The combination of the primary keys of the participating entity tables (e.g., student_id and course_id).
- Because one student can be linked to many courses and one course can be linked to many students. A single column cannot store all these connections properly.

Part II

Part A – Entity and Relationship Identification

- Entities and Key Attributes
 1. Customer (customer_id)
 2. Restaurant (restaurant_id)
 3. Order (order_id)
 4. MenuItem (item_id)
- Relationships

1. Customer – Order

Cardinality: 1:N

Justification: A customer can place multiple orders, but each order is placed by exactly one customer.

2. Restaurant – Order

Cardinality: 1:N

Justification: A restaurant can receive many orders, but each order is associated with exactly one restaurant.

3. Restaurant – MenuItem

Cardinality: 1:N

Justification: A restaurant can offer multiple menu items, but each menu item belongs to one restaurant.

4. Order – MenuItem

Cardinality: M:N

Justification: An order can contain multiple menu items, and a menu item can appear in many different orders.

Part B – Attribute Classification

Multivalued Attribute - Customer phone number , because a customer can have more than one phone number, so the attribute can store multiple values.

Composite Attribute - Customer Address because an address can have multiple components such as house no., street, city, state, and pincode.

Part C – Relational Schema Design

Since Order and MenuItem have an M:N relationship, a junction table is required.

Schema:

OrderMenuItem(order_id, item_id, quantity)

Primary Key - (order_id, item_id)

Foreign Keys - order_id, item_id

Additional Attribute - quantity (stores the quantity of each menu item in the order)

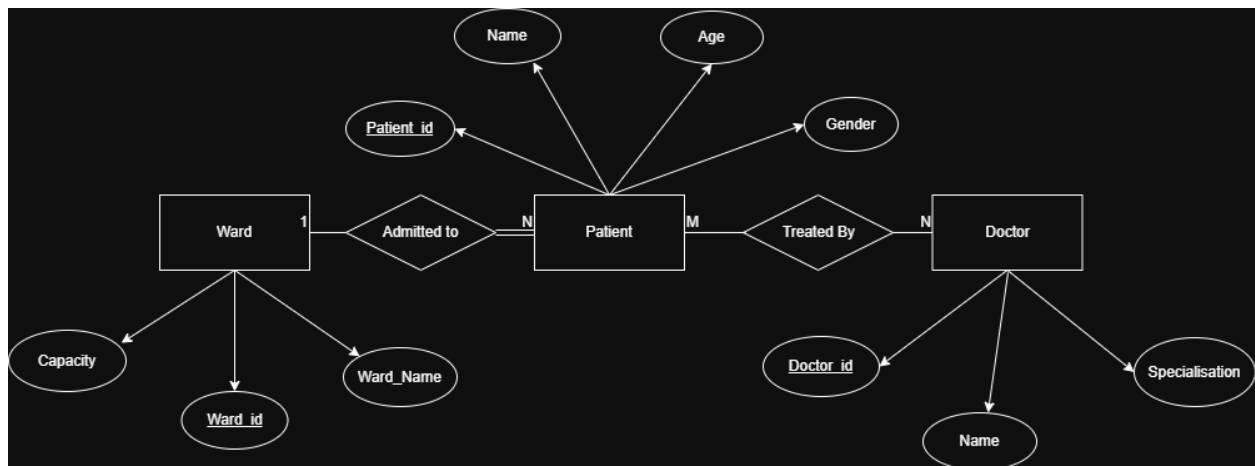
Final Schema:

OrderMenuItem(order_id, item_id, quantity)

- Primary Key: (order_id, item_id)
- Foreign Keys: order_id, item_id
- Additional Attribute: quantity

Part III

Question 1 :



Question 2:

The relationship between doctors and patients is many-to-many because one doctor can treat many patients and one patient can be treated by many doctors. To represent this relationship in a database, a separate table called Treatment can be created. This table connects doctors and patients and can also store details such as treatment date or diagnosis. The primary key of this table can be made using both Doctor_id and Patient_id together.

In the relationship between wards and patients, every patient must be assigned to a ward, so the patient has total participation. However, a ward can exist even if no patients are currently admitted to it, so the ward has partial participation. In the relationship between doctors and patients, both have partial participation because a doctor may not have any patients at a particular time, and a patient may not yet be assigned to a doctor.

One attribute decision is the patient address. Address can be treated as a composite attribute because it can be divided into smaller parts such as house number, street, city, state, and PIN code. This makes the data easier to store and manage.

